

any timeline for delivering the feature. Instead, she asks and seeks more clarity on the downstream API that will be used. She also asks if there is a sandbox environment to test the API.

Sonali tells Meera that she will loop in the team on the email received from the client. She also explains to Meera about how that email contains documents and spreadsheets relevant to the feature.

Now, as the feature requires a lot of fields (354 to be precise), Meera picks up all the 354 feeds from the sheets, uses her editing skills, and posts the status of the task as 'dev done'. Sonali, after a few days, comes back to meet Meera and informs her that the client has requested for some clarifications about the feature, and the contract. The client, Sonali informs, has also requested for minor changes. Meera leaves the project she is working on, and goes back to the bank loan feature to add and make changes as suggested by the client, but because the changes are minor, she is not quite excited about making them.

Fast forward a few months later, and the story is still not over! The testers who are checking the API realise that there are a lot more things that need to be done. They also notice the error responses that are coming from the API are different from the ones mentioned in the contract. Even the 200 'OK' responses are different, probably because of the way they are defined in the document.

This is the point where Meera gets a little irritated as the work is not interesting. She is also tired of the time the feature is taking, in addition to the blame game taking place inside the team. Hence, at this point, the arguments begin.

A lot of changes later, the team manages to push the feature ahead and, as it gets delivered, the team members start looking at charting a map to zone what did and did not go well.

A few open source specifications that can be used to define your APIs are described briefly below.

OpenAPI Specification (OAS): This specification is useful for defining REST APIs. It is a community-driven open specification within the OpenAPI Initiative, a Linux Foundation collaborative project. This specification defines a standard, programming language-independent interface description for HTTP APIs that allows both humans and computers to discover and understand the capabilities of a service without requiring access to source code.

AsyncAPI: AsyncAPI is an open source initiative that seeks to improve the current state of Event-Driven Architectures (EDA). The long-term goal of the project is to make working with EDAs as easy as it is to work with REST APIs. The AsyncAPI specification settles the base for a greater and better tooling ecosystem for EDAs. You may use this specification if you are looking for a solution to automate and formalise the documentation or code generation of your event-driven (micro)services.

JSON Schema: This allows you to validate JSON documents used in OpenAPI and AsyncAPIs. The specification is split into two parts — core and validation. The core defines the basic foundation and the validation defines the validation keywords of JSON Schema.

What were the troubles?

The first thing the team focused on was the way the contract was sent to them. It found that the contract was defined loosely, and also error-prone. The documents shared by the client made the team go back and forth a lot of times. The business manager had to constantly keep in touch with the client to seek clarification about the pieces that were missing. It turned out that the first major challenge was not coding,

but the way the two teams (client and developers) were communicating.

The integration, however, failed even after all the efforts, and the prime reasons for this, the team realised, included incorrectly spelt fields, mismatched data types, case sensitive fields, error parsing issues, missing responses, and more. In a nutshell, the errors arose as proper marking was not done by the client, and the contract was not used in the correct way.

```

RequestError:
  type: object
  properties:
    type:
      description: A URL that uniquely identifies the problem type.
      type: string
      example: "/problem/validation-error"
    title:
      description: A human readable description of the error.
      type: string
      example: "Your request parameters didn't validate"
    status:
      description: The HTTP response code.
      type: number
      example: 400
    instance:
      description: The URL of the call for which the error occurred.
      type: string
      example: "/media/analyze"
    detail:
      description: An optional field with more information depending on the type.
      type: string
      example: "job_id is required"
  
```

Figure 1: Definition of request error